

$$P(x) = \sum_{i=0}^n y_i \cdot \ell_i(x), \quad \ell_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

$$X_k = \sum_{j=0}^{n-1} a_j \cdot \omega^{jk} \pmod{p}$$

ω is an n -th root of unity in F_p

Security Review Report

NM-0853 Privacy Cash

Core EVM

March 16, 2026



NETHERMIND
SECURITY

Contents

1 Executive Summary	2
2 Audited Files	3
3 Summary of Issues	3
4 Privacy Cash	4
4.1 EtherPool	5
5 Circuit-to-Verifier Consistency Check	7
5.1 Circuit Compilation	7
5.2 Verify the final zkey	7
5.3 Verifier Contract Export and Diff	7
5.4 Conclusion	8
6 Solana/EVM Circuit Compatibility Check	9
6.1 Circuit Compilation	9
6.2 Proof of EVM Compatibility Testing	9
6.3 Conclusion	9
7 Risk Rating Methodology	10
7.1 ZK Audit Issue Classification	10
8 Issues	11
8.1 [Info] Avoid identical trees in EtherPool on different chains	11
8.2 [Info] The MerkleTreeWithHistory does not use domain separation between leaf and internal nodes	11
8.3 [Info] Unreachable code in zeros() function	11
9 Documentation Evaluation	12
10 Test Suite Evaluation	13
10.1 Tests Output	13
11 About Nethermind	15

1 Executive Summary

This document presents the security review performed by [Nethermind Security](#) for the [Privacy Cash EVM](#) smart contracts and evaluation of existing Solana ZK circuit usage on EVM from March. 10, 2026 to Mar. 12, 2026. The Ether Pool contract is a shielded pool using ZK proofs to verify the correct commitment and nullification of notes in a merkle tree.

The audited code comprises 267 lines of code written in the Solidity language. The audit included the EtherPool contract and Merkle Tree with History contract together. Additionally, the circom files were compared with the [Solana equivalent files](#) at commit `880c069bd4df95bc3921ce3b422bc1875ab7c13a` for differing constraints as well as verifying the zkey, verifier contract and EVM applicability.

The audit was performed using (a) manual analysis of the codebase and (b) automated analysis tools. **Along this document, we report** three points of attention, where they are classified as Informational. **Fig. 1** summarizes the issues identified in the *smart contracts*.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 summarizes the issues. Section 4 presents the system overview. Sections 5 and 6 present the circuit-to-verifier consistency and Solana/EVM circuit compability checks. Section 7 discusses the risk rating methodology. Section 8 details the issues. Section 9 discusses the documentation provided by the client for this audit. Section 10 presents the compilation, tests, and automated tests. Section 11 concludes the document.

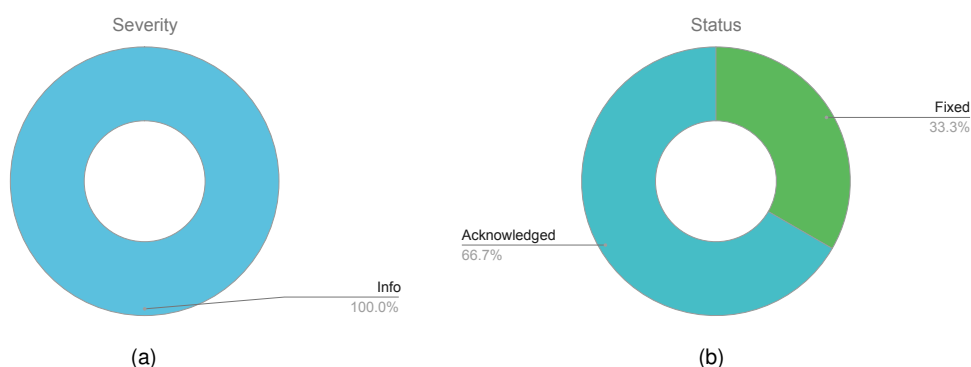


Fig. 1: Distribution of issues: Critical (0), High (0), Medium (0), Low (0), Undetermined (0), Informational (3), Best Practices (0). Distribution of status: Fixed (1), Acknowledged (2), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	Mar 12, 2026
Final Report	March 16, 2026
Response from Client	Regular responses during audit engagement
Repository	privacy-cash-core-evm
Audit Commit	ed4566d007b01a8e3672ef5787e4609b973622d9
Final Commit	f5a67581fb685f0c32a6eef200598dbf081bfdc5
Documentation	-
Documentation Assessment	Medium

Summary of Engagement

Dates	Methods	Consultants Engaged	Level of Effort
Mar. 10-Mar. 12, 2026	Manual	3	9-person days

2 Audited Files

Contract	LoC	Comments	Ratio	Blank	Total
contracts/Proxy.sol	2	1	50.0%	1	4
contracts/MerkleTreeWithHistory.sol	119	16	13.4%	18	151
contracts/EtherPool.sol	146	7	4.8%	28	180
TOTAL	267	24	7.2%	47	335

3 Summary of Issues

The table below presents a summary of issues detected in the contracts and circuits.

	Finding	Severity	Update
1	Avoid identical trees in EtherPool on different chains	Info	Acknowledged
2	The MerkleTreeWithHistory does not use domain separation between leaf and internal nodes	Info	Acknowledged
3	Unreachable code in zeros() function	Info	Fixed

4 Privacy Cash

Privacy Cash Core is a private Ether transaction protocol built on the EVM that implements the UTXO model with ZK proofs. Every interaction with the protocol (deposit or withdrawal) is routed through a single `transact()` function exposed by the `EtherPool` contract via a UUPS upgradeable proxy.

Basically, the sender locally constructs a Groth16 proof implemented in the Circom circuits, which attests to the validity of the input and output UTXOs without revealing amounts and owners, or the transaction graph. On-chain, the proof is verified by the `Verifier2` contract and the output commitments are inserted into a Poseidon-based incremental Merkle tree (`MerkleTreeWithHistory`).

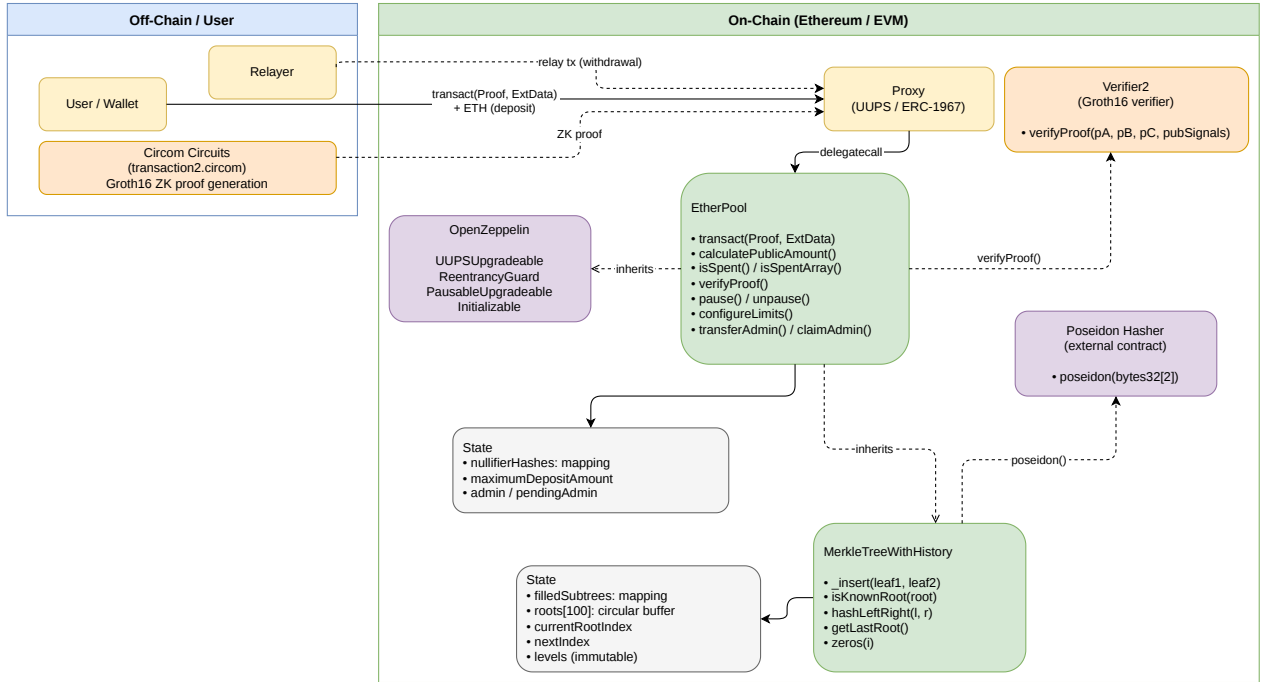


Fig. 2: Privacy Cash Core — Overview

MerkleTreeWithHistory: The `MerkleTreeWithHistory` contract implements an incremental Merkle tree based on the Poseidon hash function, designed to store the *commitments* of UTXOs deposited into the protocol. The tree has a configurable depth (up to 31 levels) and maintains a circular history of the last 100 roots, allowing recently generated proofs to remain valid even after new insertions.

Tree Structure: The tree is initialized with zero leaves whose precomputed hashes are *hard-coded* in the contract (function `zeros(i)`). Each level i of the tree has a canonical empty leaf value z_i , where:

$$z_0 = 0x062e4bce. . . , \quad z_i = \text{Poseidon}(z_{i-1}, z_{i-1})$$

The core state variables are:

- `filledSubtrees[i]`: hash of the most recent complete subtree at level i , used to compute the new root without storing the entire tree.
- `roots[k]`: circular buffer of size `ROOT_HISTORY_SIZE = 100` containing historical roots.
- `currentRootIndex`: index of the most recent root in the buffer.
- `nextIndex`: index of the next available leaf.

Pair Insertions: The internal function `_insert(leaf1, leaf2)` always inserts two *commitments* simultaneously, corresponding to the two outputs of each `JoinSplit` transaction. The hash of the pair is computed at the base and propagated upward to the root:

$$h_0 = \text{Poseidon}(\text{leaf1}, \text{leaf2})$$

At each level i , h_{i-1} is combined either with the filled subtree on the left (if the position is odd) or with the canonical zero value z_i (if it is even). The new root is stored in the circular buffer and `nextIndex` is incremented by 2.

Root Verification: The function `isKnownRoot(root)` iterates through the circular buffer from the most recent to the oldest entry, accepting any root present in the history. This guarantees concurrency tolerance: transactions that generated their proof against a slightly outdated root are still accepted as long as the root remains within the 100-insertion window.

4.1 EtherPool

The EtherPool contract is the protocol entry point for private Ether transactions. It inherits from MerkleTreeWithHistory and implements the UUPS upgradeability pattern, as well as ReentrancyGuard and PausableUpgradeable from OpenZeppelin. Every operation — deposit, internal transfer, or withdrawal — is routed through a single function `transact()`, which validates a ZK proof before any movement of funds occurs.

UTXO Model and Cryptographic Types: The protocol operates using UTXOs (*Unspent Transaction Outputs*) that never appear in plaintext on the blockchain. Each UTXO has the following logical structure (defined in the Circom circuits):

```
UTXO {
  amount      : uint248 // value in wei
  pubKey      : Field    // Poseidon(privateKey)
  blinding    : Field    // random blinding value
  mintAddress : Field    // token address
}
```

From the UTXO two public values are derived that appear on-chain: **Commitment:** Represents the UTXO in an opaque form. It is inserted into the Merkle tree when creating an output:

```
commitment = Poseidon(amount, pubKey, blinding, mintAddress)
```

Nullifier: Invalidates a UTXO when it is consumed as an input. The circuit first computes an internal signature before deriving the nullifier:

```
signature = Poseidon(privateKey, commitment, merklePath)
nullifier = Poseidon(commitment, merklePath, signature)
```

where `merklePath` is the index (position) of the leaf in the tree. Binding the signature to the position ensures that the same commitment cannot be spent using different paths.

Contract Data Structures: The `transact()` function receives two arguments:

```
struct ExtData {
  address recipient      // withdrawal recipient
  int256 extAmount      // external value (+deposit / -withdrawal)
  address feeRecipient   // relayer address
  uint256 fee           // relayer fee
  bytes encryptedOutput1 // encrypted output UTXO for the receiver
  bytes encryptedOutput2 // encrypted output UTXO for the receiver
}

struct Proof {
  uint[2] pA             // Groth16 proof - G1 element
  uint[2][2] pB          // Groth16 proof - G2 element
  uint[2] pC             // Groth16 proof - G1 element
  bytes32 root           // Merkle tree root used in the proof
  bytes32[2] inputNullifiers // nullifiers of the 2 inputs
  bytes32[2] outputCommitments // commitments of the 2 outputs
  uint256 publicAmount   // extAmount - fee (sign encoded)
  bytes32 extDataHash    // hash of the external data
}
```

The `extDataHash` field binds the external data to the ZK proof:

```
extDataHash = keccak256(abi.encode(ExtData)) mod p
```

where p is the BN254 field size (`FIELD_SIZE`). This prevents a relayer from modifying the recipient or the fee without invalidating the proof.

The `transact()` function

This is the core function in the contract. It performs several checks before modifying any state:

- Valid root:** `isKnownRoot(args.root)` — the root used to generate the proof must exist in the history.
- Unspent nullifiers:** both `inputNullifiers` must not exist in `nullifierHashes`.
- Correct external hash:** `extDataHash` must match the hash of the provided `ExtData`.
- Consistent public amount:** `publicAmount == extAmount - fee`, with withdrawal encoded as $p - |\text{publicAmount}|$.

- e. *Valid ZK proof:* `verifier2.verifyProof()` verifies the Groth16 proof with the 7 public inputs: root, publicAmount, extData-Hash, two nullifiers, and two commitments.

Administration and Upgradeability: The contract follows the UUPS pattern. Administrator changes follow a two-step mechanism (`transferAdmin` + `claimAdmin`), preventing accidental transfers to invalid addresses. The admin can also pause/unpause the contract and reconfigure the maximum deposit limit via `configureLimits()`.

5 Circuit-to-Verifier Consistency Check

To verify that the on-chain Verifier2.sol contract faithfully represents the intended zero-knowledge circuit, we performed a three-step consistency check spanning the full compilation pipeline: from the Circom source to the Solidity verifier.

5.1 Circuit Compilation

The circuit transaction2.circom was compiled from source using circom with the bn128 curve target, producing a fresh R1CS constraint system, a witness-generation WASM module, and a symbol file:

```
circom circuits/transaction2.circom \
--r1cs --wasm --sym \
-l node_modules \
-o build/circuits-test
```

The compiler reported the following circuit parameters:

Parameter	Value
Template instances	292
Non-linear constraints	15,932
Linear constraints	17,866
Total wires	33,862
Labels	50,476
Public inputs	7
Private inputs	67
Public outputs	0

The 7 public inputs correspond to the signals explicitly declared public in transaction2.circom, instantiated as Transaction(26, 2, 2) with nIns = 2 and nOuts = 2. Each scalar signal contributes one field element, while array signals contribute one field element per entry: root (1), publicAmount (1), extDataHash (1), inputNullifier[nIns] (2), and outputCommitment[nOuts] (2), totalling 7 field elements.

5.2 Verify the final zkey

As a final consistency check, we verify the proving key against both the freshly compiled R1CS and the Powers of Tau transcript from the Hermez Hermez Powers of Tau transcript (powersOfTau28_hez_final_18.ptau, we used the same as used for Solana). This step verifies zkey file contributions and verify that matches with the new compiled transaction2.r1cs and ptau.

```
snarkjs zkey verify transaction2.r1cs \ // @audit the fresh r1cs generated from the transaction2.circom
powersOfTau28_hez_final_18.ptau \
transaction2.zkey // @audit zkey in the repository
```

snarkjs computed the same circuit hash as in the [Trusted setup markdown](#) from the R1CS:

```
95a3ebe6 51cf2567 92d2eee4 67ce4857
3c2c4f71 4644bb19 5581706b 1adf23a9
bcb7a073 e6e8ec24 0be09fec 12744f94
71a28551 5c976862 da704d3b 7898482a
```

This matching confirms that the key was generated from the same circuit currently present in the repository. The trusted setup recorded four contributions from different participants. The command output with a confirmation:

```
[INFO] snarkJS: ZKey Ok!
```

5.3 Verifier Contract Export and Diff

The Solidity verifier was re-exported from the proving key:

```
snarkjs zkey export solidityverifier \
transaction2.zkey Verifier2.sol
```

A byte-level diff between the freshly generated file and the Verifier2.sol already present in the repository revealed a single difference:

```
+contract Groth16Verifier {
---
- contract Verifier2 {
```


The only discrepancy is the contract name, which was renamed from the `snarkjs` default (`Groth16Verifier`) to `Verifier2`. All constants, including the α , β , γ , and δ verification key points and the 8 input commitment points $IC_0 \dots IC_7$, are identical between the two files.

5.4 Conclusion

The three-step check passed in full. The circuit source code compiles to an R1CS whose hash matches the one embedded in the proving key that generates a Solidity verifier identical to the one provided in the repository, except the contract name. The `Verifier2.sol` correctly encodes the verification key derived from `transaction2.circom` and its associated trusted setup.

6 Solana/EVM Circuit Compatibility Check

To verify that the on-chain Verifier2.sol contract and transaction2.circom circuit works equivalently to the Solana implementation in the [Privacy Cash Solana Repository](#) at commit 880c069bd4df95bc3921ce3b422bc1875ab7c13a, we performed a two-step check verifying the usage of the ZK Circuit on the EVM.

6.1 Circuit Compilation

As stated above, the circuit transaction2.circom was compiled from source using circom with the bn128 curve target, producing a fresh R1CS constraint system, a witness-generation WASM module, and a symbol file:

```
circom circuits/transaction2.circom \  
--r1cs --wasm --sym \  
-l node_modules \  
-o build/circuits-test
```

6.2 Proof of EVM Compatibility Testing

To ensure the correctness of the Zero Knowledge Proof (ZKP) in the context of the Ethereum Virtual Machine (EVM), we designed and implemented a suite of tests. These tests were intended to thoroughly verify the ZKP's behavior in various scenarios, ensuring that it passes under correct conditions and fails as expected when there are errors or tampering. The key test cases included:

- A valid Zero Knowledge Proof passes
- An invalid ZKP fails
- A valid ZKP fails if the public inputs have been tampered with

The output of running the tests was:

```
> privacy-cash-core-evm@1.0.0 test:verifier  
> npx hardhat test test/evm-compat.test.js  
  
No need to generate any newer typings.  
  
Verifier  
  Should verify a valid ZKP (1995ms)  
  Should return false for a tampered proof (1074ms)  
  Tampered public signals  
    Should return false for a valid proof with tampered Public Signal: root (1020ms)  
    Should return false for a valid proof with tampered Public Signal: publicAmount (1011ms)  
    Should return false for a valid proof with tampered Public Signal: extDataHash (1050ms)  
  
5 passing (6s)
```

6.3 Conclusion

The tests have successfully passed, as demonstrated above, providing strong evidence of the circuit's compatibility across EVM chains.

7 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Likelihood		
		Low	Medium	High
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract and circuit development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

7.1 ZK Audit Issue Classification

ZK protocols must adhere to the three principles of zero-knowledge: **Completeness**, **Soundness** and **Zero-Knowledge**. The table below describes the classification of issues in ZK circuits with their corresponding principles of zero-knowledge.

Property	Description	Classification
<i>Completeness</i>	A valid proof must always pass with an honest verifier	Over-constrained
<i>Soundness</i>	An invalid proof should pass only with negligible probability	Under-constrained
<i>Zero-Knowledge</i>	The proof must not reveal any information about the witness	Leakage

8 Issues

8.1 [Info] Avoid identical trees in EtherPool on different chains

File(s): [EtherPool.sol](#)

Description: The EtherPool contract may be vulnerable to proof and nullifier reuse if instances deployed on different chains maintain identical Merkle trees. If the state of the trees is the same, a valid proof and nullifier used on one chain could be replanted and successfully executed on another. This risk is particularly relevant if EtherPool is deployed simultaneously across multiple EVM networks and a user submits identical data to each, resulting in synchronized tree states.

Concretely, two scenarios enable this attack:

1. *Initial state:* both chains start with `roots[0] = zeros(levels)`. A deposit proof generated against the empty root is valid on any chain with that same empty root;
2. *Identical state:* if the user made the same deposits on both chains, the trees may share the same root;

Recommendation(s): Although this scenario is improbable, it remains a technical possibility. To mitigate this, consider depositing a small, unique amount into each pool using distinct blinding factors to ensure the Merkle trees diverge immediately. A more robust programmatic solution is to incorporate the chain ID into the `extDataHash` and verify it within the `transact()` function using `block.chainid` to ensure proofs are domain-bound to a specific network.

Status: Acknowledged

Update from the client: The first tx must be a deposit, and the `transact()` explicitly checks `msg.value == uint256(_extData.extAmount)`. Even if the user forge the proof, he has to deposit the same amount as in the proof, thus there won't be any loss of fund even if he forge the proof. There's no way we will launch simultaneously on multiple chains.

8.2 [Info] The MerkleTreeWithHistory does not use domain separation between leaf and internal nodes

File(s): [MerkleTreeWithHistory.sol](#)

Description: The `hashLeftRight` function uses Poseidon hash with the same calling convention for both leaf-level hashing and internal node hashing:

```

1 // leaf level (line 60)
2 bytes32 currentLevelHash = hashLeftRight(_leaf1, _leaf2);
3
4 // internal nodes (line 73)
5 currentLevelHash = hashLeftRight(left, right);

```

Without a domain tag or prefix to distinguish the two contexts, the tree is susceptible to a second-preimage attack: an attacker who can insert an arbitrary leaf value `L` equal to `Poseidon(A, B)`, where `A` and `B` are valid internal node hashes, creates an ambiguity where the same value exists simultaneously as a leaf and as an internal node. A verifier that does not enforce strict tree depth can be tricked into accepting a forged inclusion proof.

In the current EtherPool deployment this risk is mitigated by: (1) the ZK circuit enforcing a fixed tree depth of 26 levels, and (2) the circuit requiring a valid commitment preimage (`amount`, `blinding`, `privkey`) to spend any leaf, making it computationally infeasible to exploit. However, `MerkleTreeWithHistory` is a standalone, inheritable contract. Any future consumer that performs Merkle inclusion verification without those ZK constraints inherits this vulnerability.

Recommendation(s): Introduce domain separation so that leaf hashes and internal node hashes are computed in distinct domains.

Status: Acknowledged

Update from the client: It has no security impact, given the contract is used by our privacy pool with fixed merkle tree height both in contract and circuit.

8.3 [Info] Unreachable code in zeros() function

File(s): [MerkleTreeWithHistory.sol](#)

Description: The constructor enforces `_levels < 32`, meaning the maximum value for levels is 31. The function `zeros()` is called with `i == levels` in `_initialize()` and with `i` up to `levels - 1` in `_insert()`, so the maximum index ever passed to `zeros()` is 31. The entry for `i == 32` at line 146 is therefore unreachable dead code.

Recommendation(s): Remove the `i == 32` entry from `zeros()` to keep the code consistent with the enforced invariant.

Status: Fixed

Update from the client: Fixed in [PR #7](#).

9 Documentation Evaluation

Software documentation refers to the written or visual information describing software's functionality, architecture, design, and implementation. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Protocols can use various types of software documentation for their circuits. Some of the most common types include:

Circuit Specifications: A documentation that outlines the purpose, functionality, and constraints of the circuit:

- i) High-level description - What the circuit is designed to achieve, e.g., proving membership in a Merkle tree, verifying a signature, etc. Description about inputs and outputs (public and private signals);
- ii) Mathematical model - Description of the mathematical computations and proofs being implemented. In addition, describe the cryptographic primitives that are used, e.g., Poseidon hash, EdDSA verification.
- iii) Expected behavior - Present a detailed explanation of how the circuit behaves under normal, edge, and adversarial conditions.

Documentation of the Input and Output: The documentation should describe details of the inputs and outputs of the circuit.

- i) Public inputs: List of public inputs and their expected values and ranges. Also, clarify constraints on the public inputs (e.g., nonzero).
- ii) Private inputs: List of private input signals and their roles in the circuit. The documentation should also include validation requirements for private inputs within the circuit.
- iii) Outputs: A description of the outputs, their intended use and their relationship with the inputs.

Performance Metrics: Describe details on the prover and verifier performance such as, time taken to generate proofs, memory usage during proof generation, time taken to verify proofs, etc.

Testing Documentation: Testing documentation is a document that provides information about how the circuits were tested. It includes details on the test cases that were used, coverage metrics showing which parts of the circuit have been tested, and any issues that were identified during testing.

Technical whitepaper: A technical whitepaper is a comprehensive document that describes the design and technical details of the protocol. It includes information on the purpose of the protocol, contracts, its architecture, its components, and how they interact with each other;

Summary: These types of documentation are essential for circuit development and maintenance. They help ensure that the circuit is properly designed, implemented, and tested. The documentation provides a reference for developers who need to modify the circuit in the future.

Remarks about the Privacy Cash EVM documentation

The **Privacy Cash EVM** protocol documentation was made available through the code comments and through extensive code walkthrough. Furthermore, the **Social Panda** team thoroughly addressed all questions and concerns raised by the **Nethermind Security** team, providing valuable insights and a comprehensive understanding of the project's technical aspects.

10 Test Suite Evaluation

Running tests require modification. The Hasher's bytecode and ABI contract currently is generated through circomlibjs and added to the artifacts. To run the tests the contract must be deployed in testing scripts from the bytecode and ABI.

10.1 Tests Output

```
>> yarn test
yarn run v1.22.22
$ npx hardhat test

EtherPool
  should configure (292ms)
  encrypt -> decrypt should work
  constants check
BigNumber.toString does not accept any parameters; base-10 is assumed
  should deposit, transact and withdraw (5696ms)
  should deposit and withdraw with relayer fee (3194ms)
  should deposit and withdraw with zero fee (2896ms)
  attacker cannot frontrun withdraw by replacing recipient (2790ms)
  should reject tampered publicAmount (1361ms)
  should reject tampered fee (1618ms)
  should reject double spend (4665ms)
  should reject deposit exceeding limit (1423ms)
  non-admin cannot configure limits
  should reject direct ETH transfers
  admin can transfer admin to new address
  non-admin cannot transfer admin
  non-pending admin cannot claim admin
  cannot transfer admin to zero address
  admin can overwrite pending admin before claim
  new admin can configure limits after transfer
  cannot initialize twice
  should deposit and withdraw full amount, then deposit again (5928ms)
  can deposit after increasing limit (2793ms)
  cannot deposit after decreasing limit (2761ms)
  withdrawal has no limit (4145ms)
  arithmetic overflow protection (1363ms)
  should deposit with zero fee and withdraw with positive fee (2737ms)
  should reject incorrect ETH value on deposit (1378ms)
  should reject withdraw to zero address (2783ms)
  should reject unknown root (1698ms)
  should reject zero root (1560ms)
  should reject wrong extDataHash (1579ms)
  should reject zero extAmount (no shielded transfers)
  should reject zero extAmount with positive fee
  should reject deposit where fee equals extAmount
  should reject deposit where fee exceeds extAmount
  should reject withdrawal where fee equals extAmount
  should reject withdrawal where fee exceeds extAmount
  should accept deposit where extAmount exceeds fee
  should accept withdrawal where extAmount exceeds fee
  should accept deposit with zero fee
  should accept withdrawal with zero fee
  admin can pause and unpause
  non-admin cannot pause
  non-admin cannot unpause
  transact is blocked when paused (1487ms)
  transact works after unpause (1588ms)
  should be compliant (3013ms)

toFixedHex negative numbers
  should produce correct length for positive numbers
  should roundtrip negative numbers through BigNumber
  should roundtrip negative numbers through toFixedHex
  should produce correct extDataHash with negative extAmount
```

```
MerkleTreeWithHistory
#constructor
  should correctly hash 2 leaves (190ms)
  should initialize
  should have correct merkle root
  should reject levels of 0
  should reject levels >= 32
#zeros
  should have consistent zero values (each level is hash of previous level pair) (93ms)
  should return correct zeros for all levels 0 through 32 (71ms)
  should revert for out of bounds index
  should match the off-chain tree zero element at level 0
#hashLeftRight
  should reject left input >= FIELD_SIZE
  should reject right input >= FIELD_SIZE
  should accept inputs just below FIELD_SIZE
  should produce the same hash for the same inputs
  should produce different hashes for different inputs
  hasher gas
#insert
  should insert and match off-chain tree (50ms)
  should update nextIndex by 2 per insertion
  should update currentRootIndex on each insertion
  should stay consistent with off-chain tree after many insertions (252ms)
  should produce unique roots for different insertions (53ms)
#tree capacity
  should revert when tree is full (169ms)
  should accept exactly capacity leaves before reverting
  should track nextIndex correctly up to capacity
#isKnownRoot
  should return last root
  should return older root
  should fail on unknown root
  should not return uninitialized roots
  should recognize the initial empty root
  should keep all roots within ROOT_HISTORY_SIZE (325ms)
  should evict oldest root after ROOT_HISTORY_SIZE insertions (2431ms)
  should wrap currentRootIndex around ROOT_HISTORY_SIZE (1889ms)

EtherPool Upgrades
  deployProxy
    should deploy via upgrades plugin and initialize correctly (591ms)
    should have correct merkle tree initial state
    should not allow re-initialization
  upgradeProxy
    should upgrade and preserve storage
    should reject upgrade from non-admin
    should allow admin to upgrade after admin transfer
```

88 passing (1m)

11 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our Cryptography Research team is dedicated to continuous internal research while fostering close collaboration with external partners. The team has expertise across a wide range of domains, including cryptography protocols, consensus design, decentralized identity, verifiable credentials, Sybil resistance, oracles, and credentials, distributed validator technology (DVT), and Zero-knowledge proofs. This diverse skill set, combined with strong collaboration between our engineering teams, enables us to deliver cutting-edge solutions to our partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

Learn more about us at nethermind.io.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.